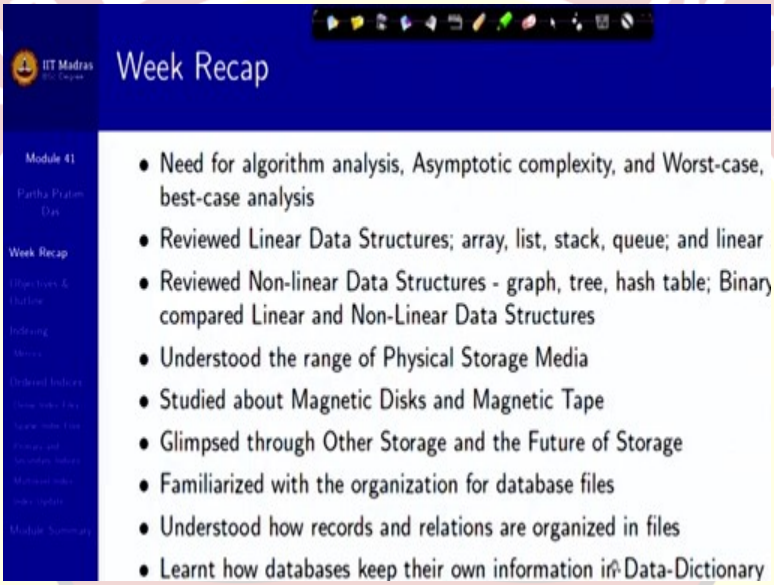
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Indexing and Hashing/1: Indexing/1

Welcome to module 41, week 9 of Database Management Systems course in the IIT Madras, online B.Sc. program.

(Refer Slide Time: 0:30)

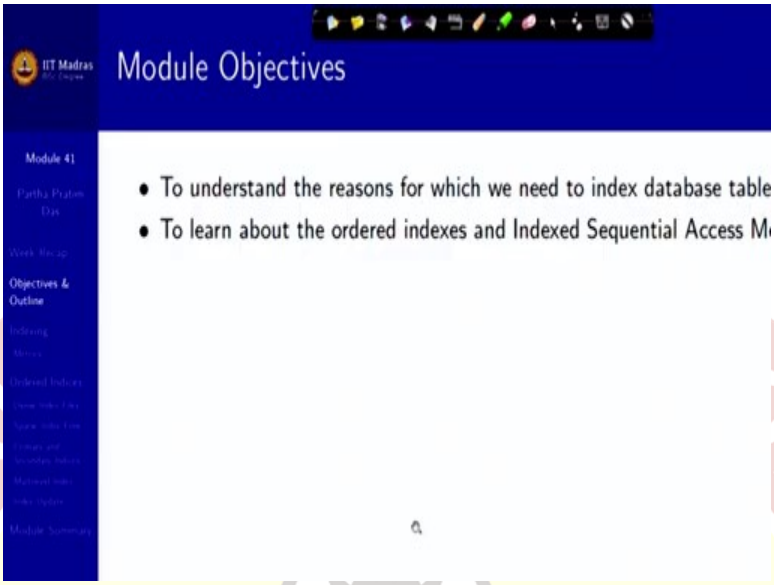


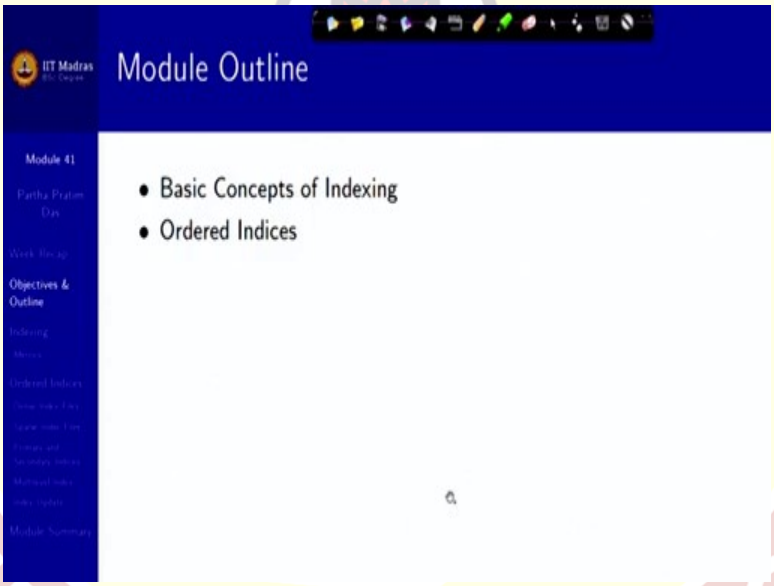
In the whole of last week, we had taken a step back to first discuss about various fundamentals of algorithms, complexity, linear and non-linear data structure. You may be aware of all these through your algorithms course, this was just to create a baseline.

And then we move forward to discuss about different physical storage media that are available for implementing the database physical layer and we observe that particularly magnetic disks are very important and kind of is a default secondary storage available for the database implementation. So, all our physical layer design will be primarily focused on the structure and behavior of the magnetic disks.

We talked about other storages, the future directions, and finally familiarized with the way a database file is organized in terms of its records and how does the database managed itself through the data dictionary or systems catalog and how it does manage the buffering to improve on the access speed.

(Refer Slide Time: 1:57)





Today from this module and for most of this week we will focus on improving the search, insert and delete functionality of the databases using a mechanism called indexing. And overall, this is called the indexed sequential access mechanism 'ISAM' and we will get into different forms of indexes and slowly over the week we will expose you through different requirements of indexing and their structures and the algorithms. So, in this module we talk about the basic concepts and ordered indices.

(Refer Slide Time: 2:43)

So, what is indexing? Let us just take a simple example, consider a table as I show here a table having just two fields, the name and the phone number. Now, you can think of either name is the key or phone number is the key, let us make the simplifying assumption that the names are unique, obviously phone numbers will have to be unique.

Now, given this table and what I also show is in a sequential representation of the, of this table as we have seen record by record, what is the record number. So, this like you can think of this as a record number, you can think of this as an array index for conceptual understanding. Now, the

requirement is how do we search on a name, given a name how do I find out the appropriate record and the corresponding phone number.

So, how to find a name and then get the phone number of that person? So, if this field were sorted in increasing or decreasing order then we can do a kind of binary search on this and quickly find the name of Professor Probitra Mitra and get the phone number, but this is actually not sorted.

(Refer Slide Time: 4:27)

Moreover, suppose I in a different query want to search by phone number, I want to find who is the faculty who has this phone number 84772, 84772 which happens to be professor Prabir

Kumar Biswas. Now, the phone number is also not sorted in increasing or decreasing order, so I cannot do a binary search, I will have to go linearly. So, in both cases I have to go linearly over the fields to search the attributes to search causing an order n performance which is not acceptable.

Now, even if I assume that sorting is an option which certainly makes the subsequent insert, delete more complicated, even if I assume that I can sort, I cannot sort on both, I have to sort either on name, then this query will become efficient order log n or I have to sort on phone number then this query will become efficient but the other one will remain to be order n.

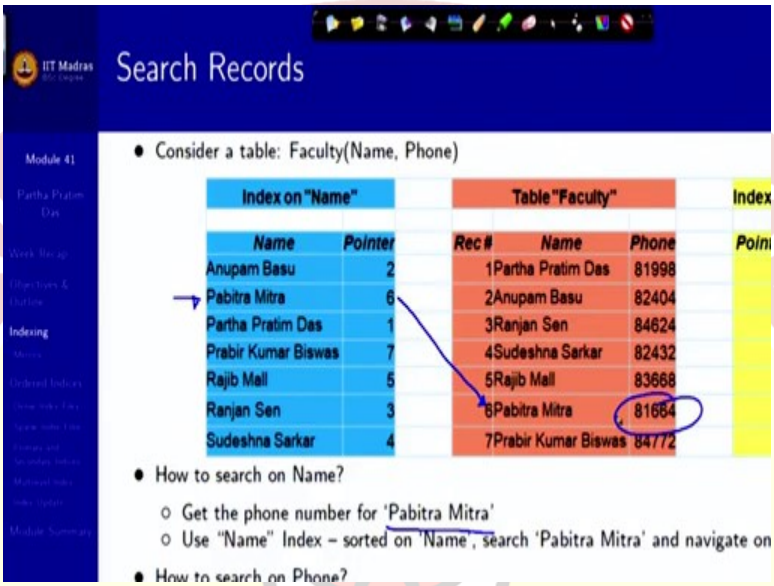
So, indexing is a mechanism by which you can solve this problem, you do not need to assume that the actual records are sorted either in name or in phone number they can be just, they can just remain arbitrary.

(Refer Slide Time: 5:42)

What you do, you create a secondary table, another table typically called an indexing table or indexing relation. Wherein you suppose if I want to consider name, I just collect the names of, I mean I just take this column, I do a projection of this column and then sort them according to the name. So, this is like pi name of faculty and then you sort them. So, here you see the names are in sorted order. So, the advantage here is using this index file I can do a binary search.

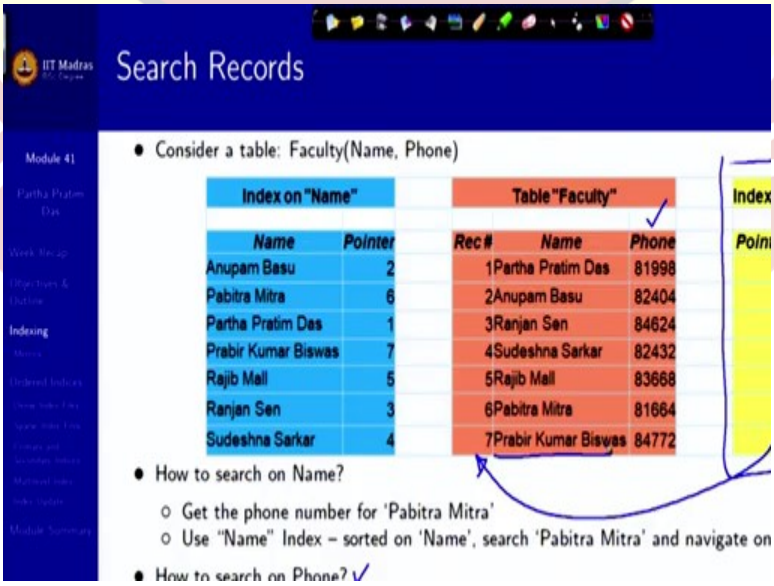
And then what do I have, associated with this I keep the record number which is like a pointer, in reality it could be a complex address expression but I am just assuming for understanding that it is pointer.

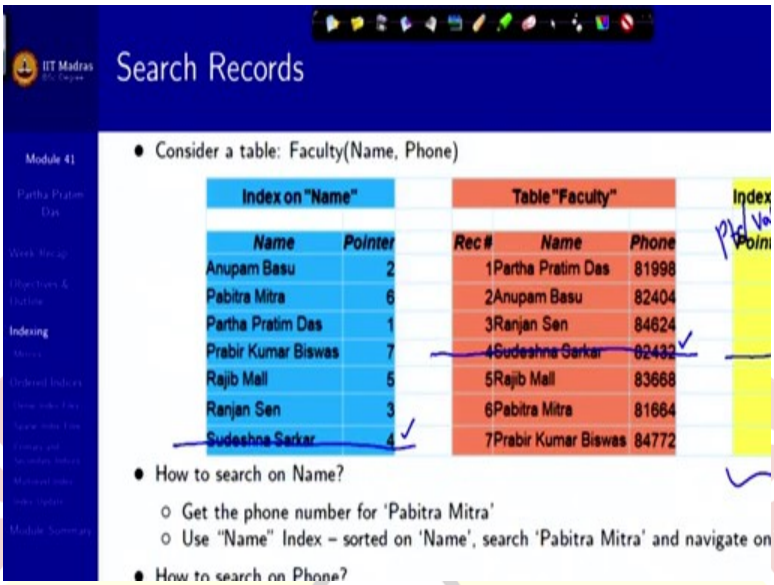
(Refer Slide Time: 6:49)



So, what will happen if I now look for Professor Pabitra Mitra, I can simply do a binary search, get this very quickly and I know that this professor, this faculty will be found in record 6, I directly go to record 6, I have the phone number.

(Refer Slide Time: 7:08)





Now, please consider that this has got nothing to do with the actual physical order of the records in the original file. So, no matter what attribute is fundamental in the particular query, if I create a corresponding index file I can always search on that particular attribute efficiently. For example, to search for phone number, I create another index file, which is taking the phone number fields, sorting them here it is in increasing order.

So, if I have to look for 84772, I have to do a binary search on this field which is easy and in log n, I will get here I have also put the associated record number, so I know it is this record, I know the name of the Professor is Prabir Kumar Biswass.

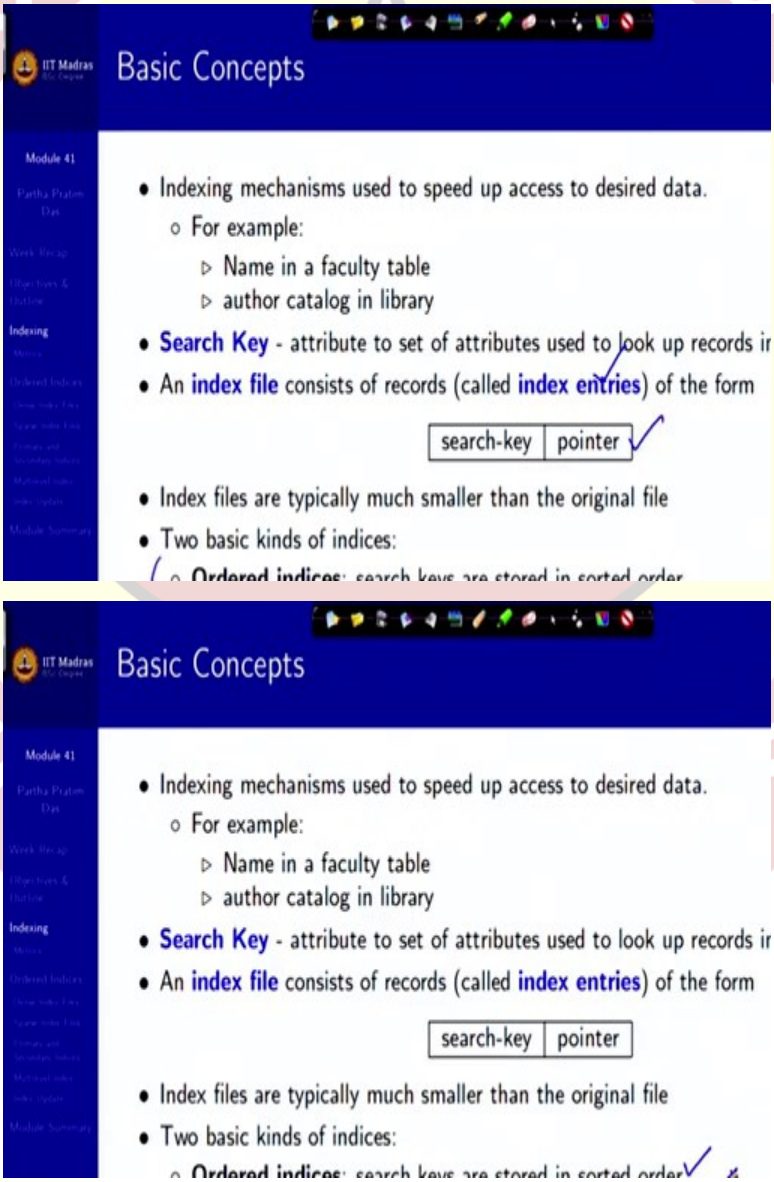
So, with this we achieve two important things, one is we do not need any specific organization of the records in the original file faculty, they can be in any order. Second is, we can choose any field, any attribute, in fact we can take multiple attributes also and create an index file so that any query that involves that attribute can be done very efficiently, this is the whole concept of indexing.

Of course, it has several overheads like I have to create this index file, so it has storage over it, if I do some insert, delete in the original faculty file, naturally I will have to update, the corresponding update will have to be done in terms of the other index files for example if I want to delete this faculty 4, then naturally this record in the index phone file and this record in the index name file, these index records, these are naturally will be called index records which have a key and the value or pointer. So, these records will be removed.

Now, removing the record here as we saw in the file structure may not be a expensive thing because I can just follow the linking strategy, we talked of chaining strategy, we can chain up on the free list and so on, it can be done efficiently. But updating on this index files is little bit more expensive because they are sorted.

So, I have to restore that sorting order, so which means that like I do in an array I have to make some movements. So, there are costs but it is more often that you try to extract information from a table, than you are updating the table, so indexing has a lot of value.

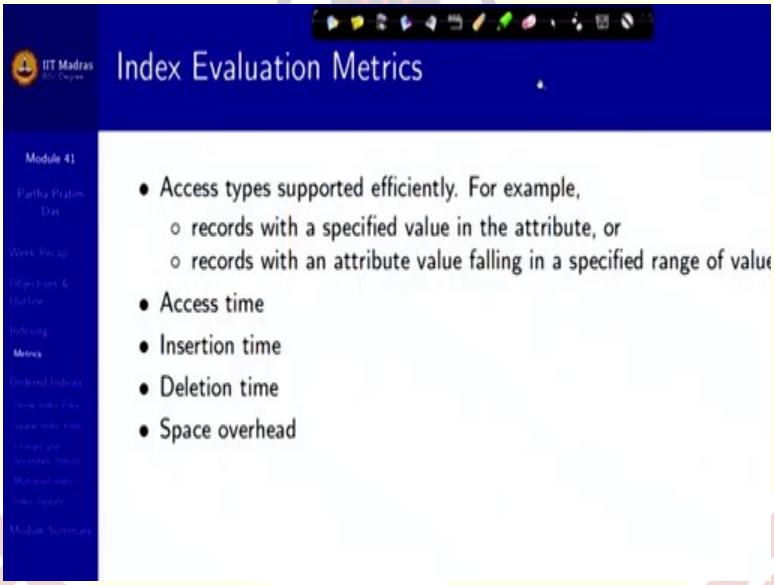
(Refer Slide Time: 10:24)



Now, the given this basic concept now you know there is a index file, there is the index entries we have seen. Now, the question is how these indexes can be maintained. So, there are two broad options used what we just showed, what we just discussed are known as ordered index, that is the index is in increasing or decreasing order of the search key, it was name in the first case, it was phone number in the second case and it has the associated issue of having more expensive insert, delete, I mean handling of insert and delete of records because it has to be re-indexed or it has to be resorted.

There is another option which is based on hashing which can make things more efficient and we will talk about that later, in the module today we will just talk about, in this module we will just talk about the ordered index.

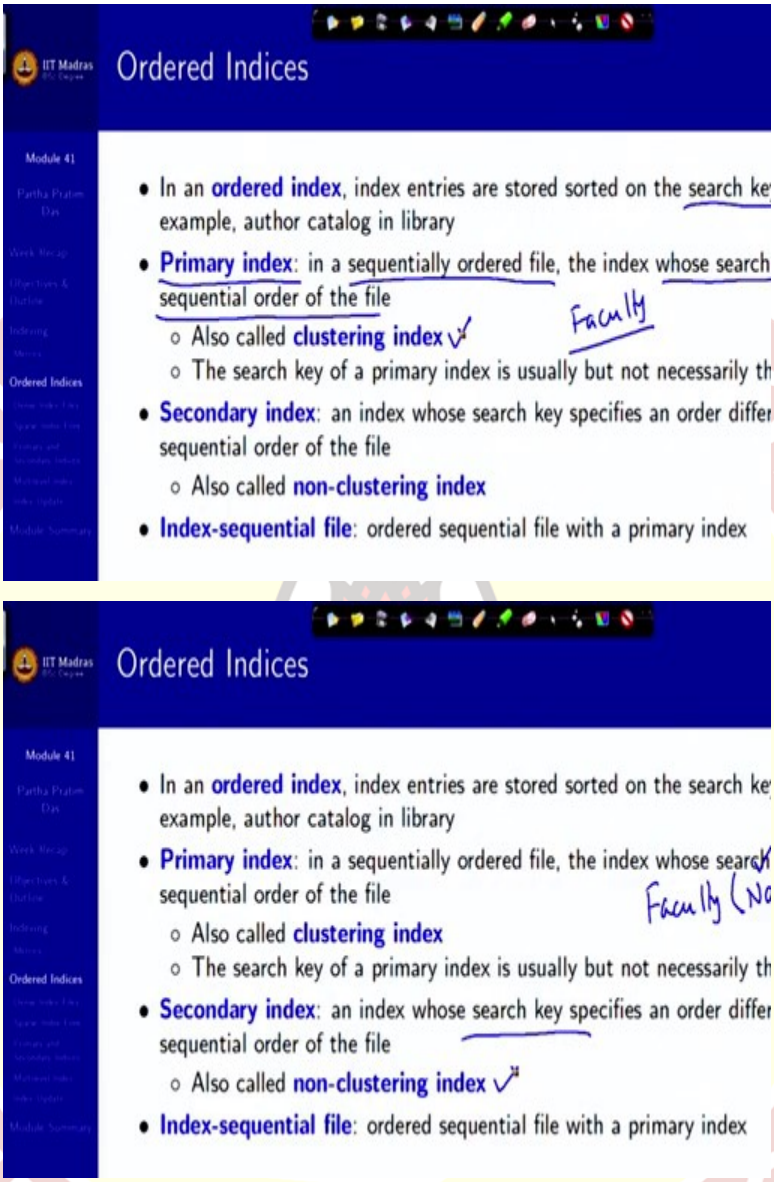
(Refer Slide Time: 11:23)



Now, naturally if you have done indexing then you would like to know what are the metrics, every time it is important to measure so that you can monitor and control. So, what are the access types that are supported efficiently, records with a specified value in attribute, records who fall in a within a specified range. There are different types of attributes, different types of data types.

So, based on that different types of queries you expect and based on that you have different types of indexing that you would like to adopt and measure accordingly. Certainly, the access time, insertion time, deletion time would be the key parameters along with the space over it which are already you have understood.

(Refer Slide Time: 12:09)



So, what is an ordered index? It is an ordered index are index entries stored on the search key value. Now, mind you this is the search key value as whatever I want to search on like I did name or I did phone number and so on. An index is called primary if in a sequentially ordered file like the faculty we saw, the index whose search key specifies the sequential order of the file, if it is a sequentially ordered file, if it is not a sequentially ordered file like the example you saw, then it does not have a primary index.

But if it is sequentially ordered, then it is ordered according to some search key, that search key is called the primary index and it is also called the clustering index. And the search key of the

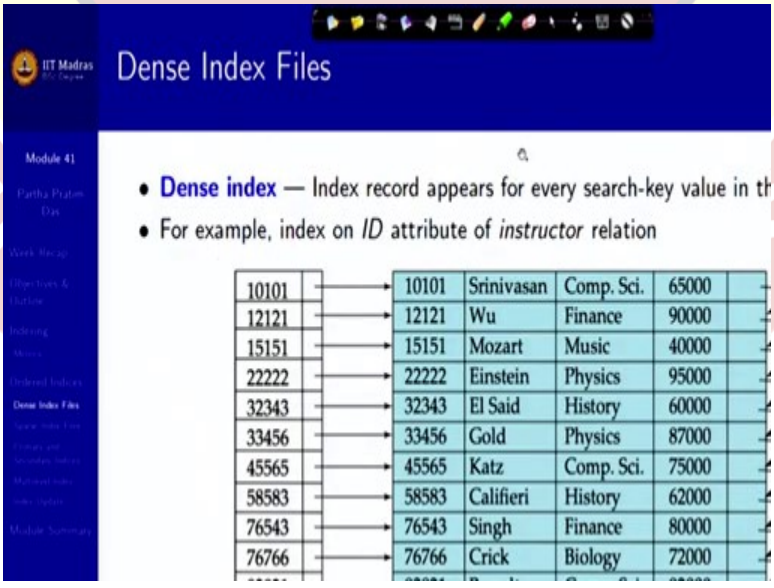
primary index is usually the primary key but may not be the primary key, it is not mandatory that I will have to organize the sequential order of the file in terms of the primary key as the primary index.

For example, primary key may have three attributes and I may have used only one of them as a primary index to order the files which consequently means that on the primary index there could be duplication of values of the index on that field because it is not a primary key. But it helps in my several search and other mechanism.

A secondary index is an index whose search key is different from the sequential order of the file. So, if I take the faculty example and I have name and I have phone number, I can, I may choose to order on this field, keep the sequential file ordered in this field which will be the primary index. And this may not be the key because names may be duplicate so name and phone number together may be the key, I do not care.

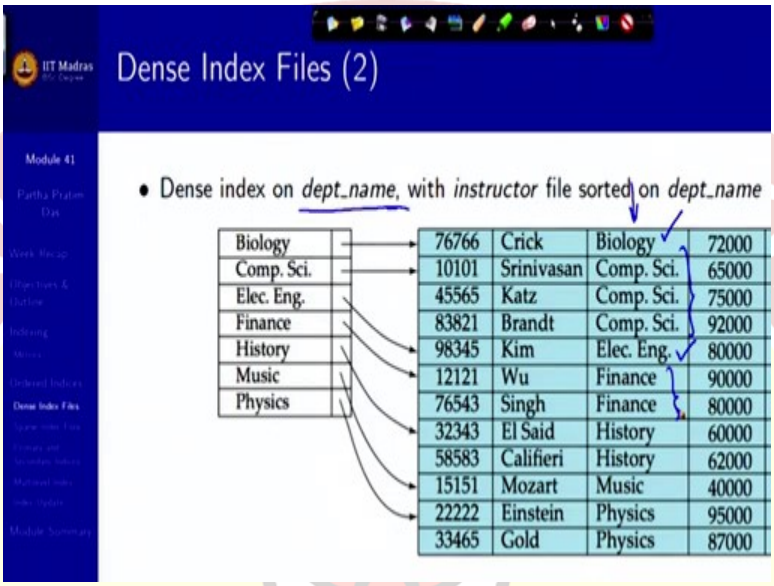
And now if I organize this file in an order which is different from the sequential order of the primary index that is suppose it is in terms of phone number, then it is called a secondary index or is called a non-clustering index. When we talk about index sequential file, you mean ordered sequential file with a primary index.

(Refer Slide Time: 14:53)



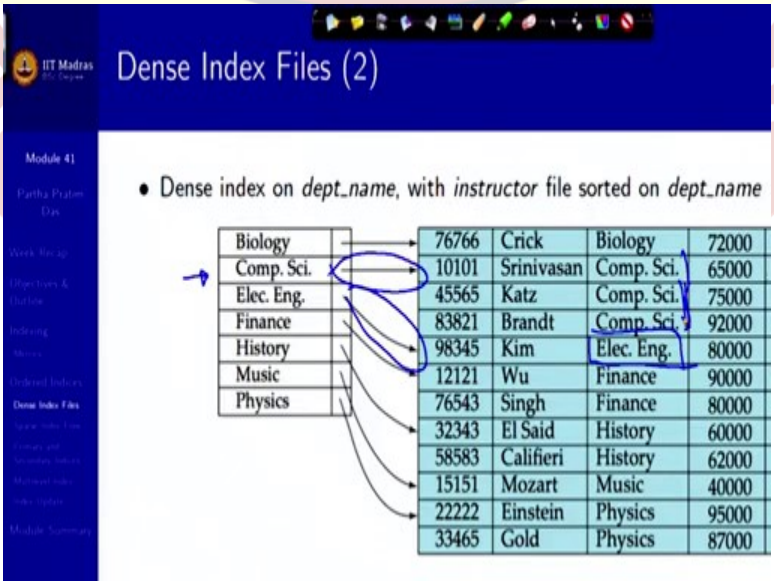
So, that is a basic idea now the index could be dense. For example, if you involve all the key values of the search key then you call it call it a dense index. For example, this is in, this has a primary index of ID of the instructor which also happens to be the key.

(Refer Slide Time: 15:14)



But it is not necessarily that it will have to be a key, I can have sorted this with a dense index on department name, you know department name is not a key here, of the instructor file here. So, all biology faculty instructors come here, all computer science come here, all finance come here and so on and they are sorted in increasing order.

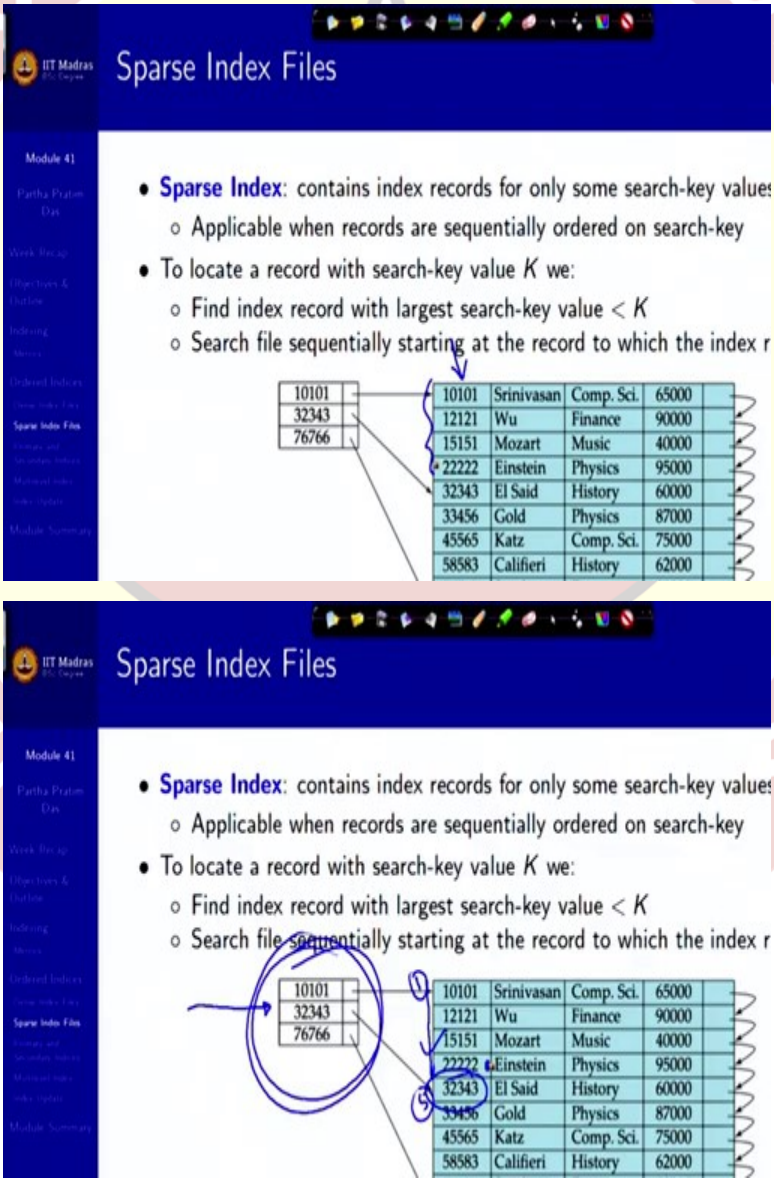
(Refer Slide Time: 15:42)



And when I create this index on them, naturally there are three records all of which have computer science. So, I can all of them have identical values. So, from this index I have only one pointer going to the first record. And then I have Electrical Engineering which has this record so the pointer goes to that record.

So, the records in between are certainly in Computer Science till I hit the next index which is Electrical Engineering. So, on a secondary index things could also look like this when you have duplicate values.

(Refer Slide Time: 16:21)



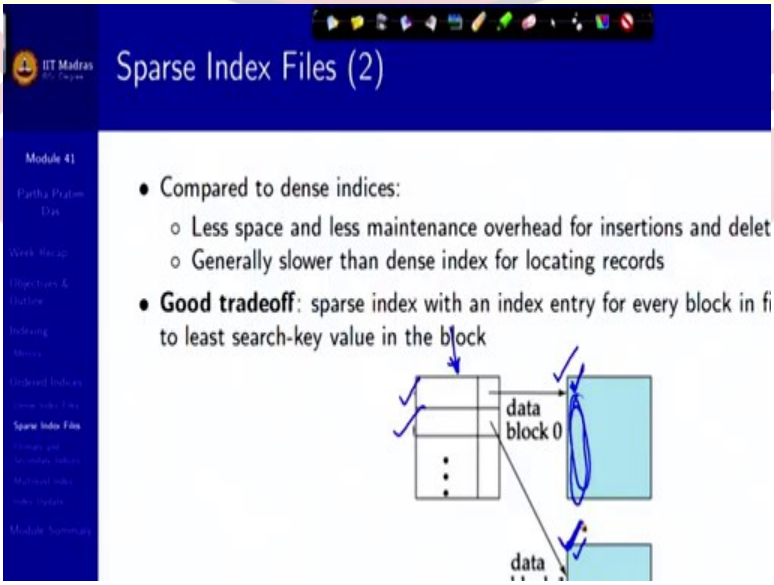
Now, dense index, the problem of dense index is it could be very large and you may want to make a trade-off between how much index space you want to keep and how much index management you want to do, vis-à-vis the absolute access time. So, what you could do is, you could make a sparse index, sparse index means that you do not keep all the index values of the search key but you keep one skip a few, keep one skip a few.

So, you are still thinking about a primary sorted index here, but it is sparse in that you are not keeping like the dense index all these index records. Instead you are just keeping the first and then you are keeping 1, 2, 3, 4, fifth one, you keep the first one, you keep the fifth one, 5, 6, 7, 8, 9 you keep the tenth one and so on, so you are keeping say 1 out of 5.

Which means this file will be one fifth in size and therefore its maintenance will be easier and so on. Of course your access strategy will change because if you now get a value 15151, you may not find it in the index file, if you find its fine, if you take 10101 you will find it directly will go to that record, if you do not, then the binary search will tell you what are the two values between which it is expected, this is the failure point of this search.

So, you have to search the records from the previous index to the next. So, you keep on searching here till you come to this index which you know will be more than this and you will be able to find this. So, if you had 17293, you will keep on searching and at this point you will come to the conclusion that this record does not exist. So, that is the beauty of the sparse index.

(Refer Slide Time: 18:22)



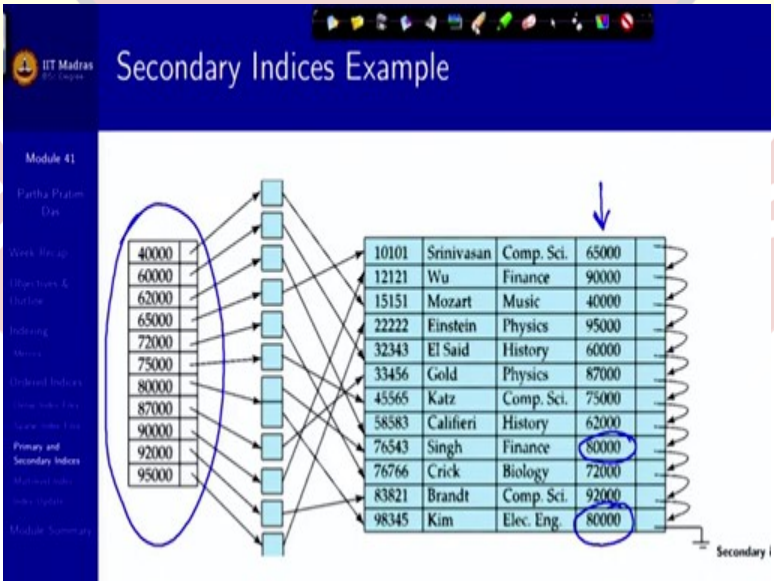
And sparse index is very heavily used, it is a good tradeoff between how much you have to manage as additional data and work on them and how much efficiency of access you get. For example, you can have a sparse index based on the blocks. You know we have already said that the data is always exchanged between the hard disk and the memory in order of block. So, even you have a block you have entire of that together.

Now, you have multiple blocks for a file. Now, what you can do is you could have the first record of every block, the in terms of the primary index the first record of each of that as your index entry. So, you have this and the next one go. So, first what you are deciding when you search on this index what you get to see is the likely block where your data is expected.

So, you have the first and the remaining all records are greater than this but less than this and they are in order, so then you can or they may not even be in order, as long as you maintain that all of them are greater than equal to this and less than equal to this, you will do a linear search within the block.

So, it, one way it cuts down in terms of finding the block quickly and on other way you pay some penalty by searching through the block through a linear mechanism or you can do some other stuff as well.

(Refer Slide Time: 19:53)

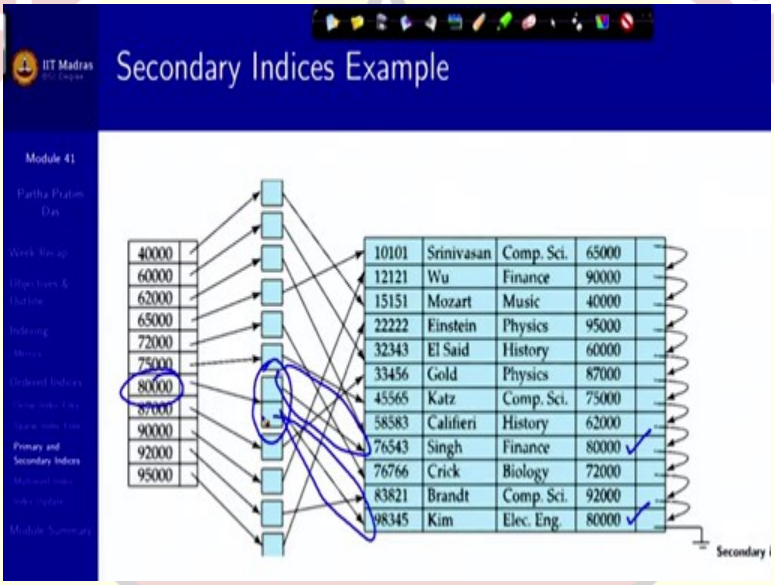


There is another concept called the secondary index because so far the indexes that we have shown always gives you one indexed record but it is not necessary because you are indexing on

any attribute and there could be multiple values. So, how do you handle that? Because if you create an index, for example, here which is the salary, if you create an index on the salary you will find that you will get two entries which are identical.

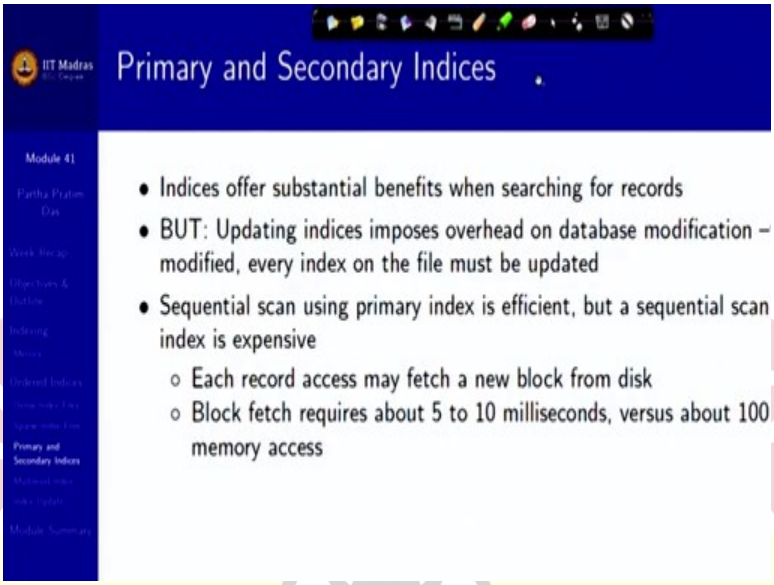
So, how do you handle that? Naturally, you may actually have several entries which are identical. So, one way is to allow identical entries on your index file but what is more efficient is you create an index with the unique values and the pointer does not directly point to the specific records. What it does it points to a list which in turn points to all the records having that value.

(Refer Slide Time: 21:06)



So, you can see the example specifically here 80000 is a repeated value, so it has, it occurs in two places. So, what you have when you come from 80000, you have a list of two indexes, one that takes you to this record this one and the other that takes you to the other record. So, secondary index is an index on an index kind of but, or kind of specifically to take care of your multiple values on an attribute which is your search key, necessarily secondary indices will have to be dense.

(Refer Slide Time: 21:46)

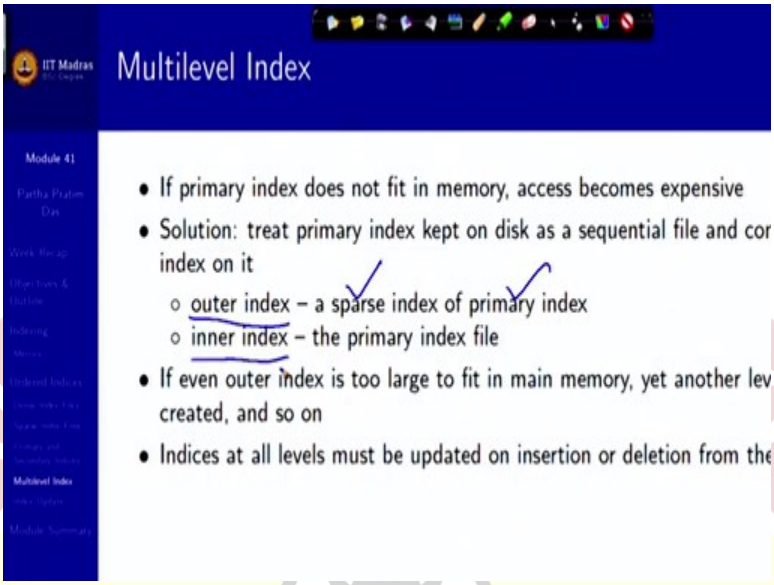


So, we have primary and secondary indices which offer substantial benefits when searching for the records but naturally updating indices imposes the overhead on database modification as I explained in the very beginning. When the file is modified every index on that file must be updated.

So, you might, what happens with the young developers of databases looking for a lot of performance tend to make a lot of index files, on different tables and different attribute groups and so on without really thinking what is going to be the access pattern, what is going to be the typical queries.

Now, the problem with that is your accesses may be, may get important, may get fast but you pay huge penalty in terms of the updates, in terms of values getting updated or records getting inserted or deleted. So, sequential scan using primary index is efficient but sequential scan using secondary index is expensive, each access may fetch a new block from the disk, so all these factors you will have to keep in mind, play around with and basically trade off with when you actually design the indexes based on your query pattern.

(Refer Slide Time: 23:04)



The other concept that often is involved is multi-level index. See, you are indexing and you want to search that entire thing. So, you would be able to do a binary search, it is in memory remember always that, your blocks are in disk, so unless you bring them to the memory on the buffer you cannot directly search on them. So, you want to search through the index.

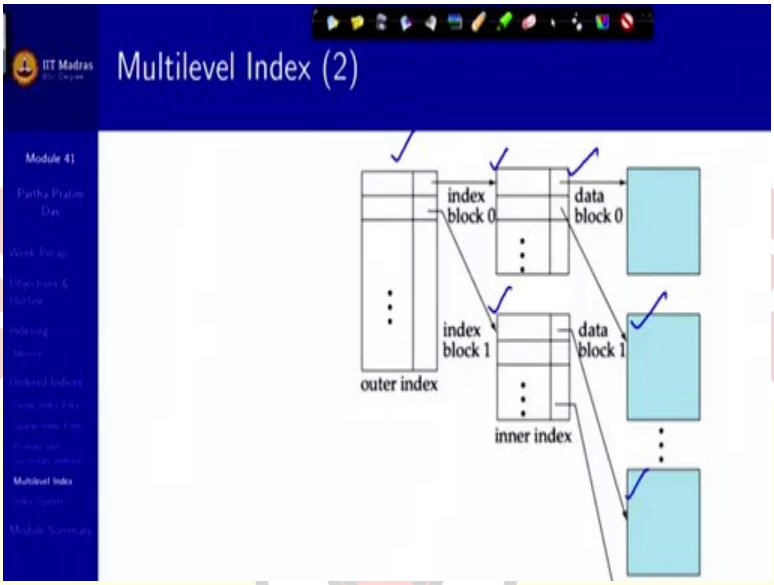
Now, the index file needs to be brought into the memory so that you can do a binary search, otherwise it will involve lot of cost. Now, suppose the index file grows so big that it does not fit into one block, what do you do? So, the idea is simple, why are you doing this indexing? Because your entire file, relation that you are keeping in that file is not fitting into a block, so you are indexing.

So, if your index does not fit into a block, what do you do? You create an index on the index, index on the index, so and that index that you are creating on the index should be a sparse index that is the reason you need a parse index. Because if you keep index are expectedly unique values. So, if you keep do an index on that, keeping everything then you do not benefit in the actual file you benefit because your other attributes are also there, so there is a lot of space taken by them.

In an index file there is nothing else it is just the search key and the pointer. So, if you want to make another index on this that should be a sparse index on the primary index and that quite naturally is called an outer index, the original primary index file is called the inner index. Even

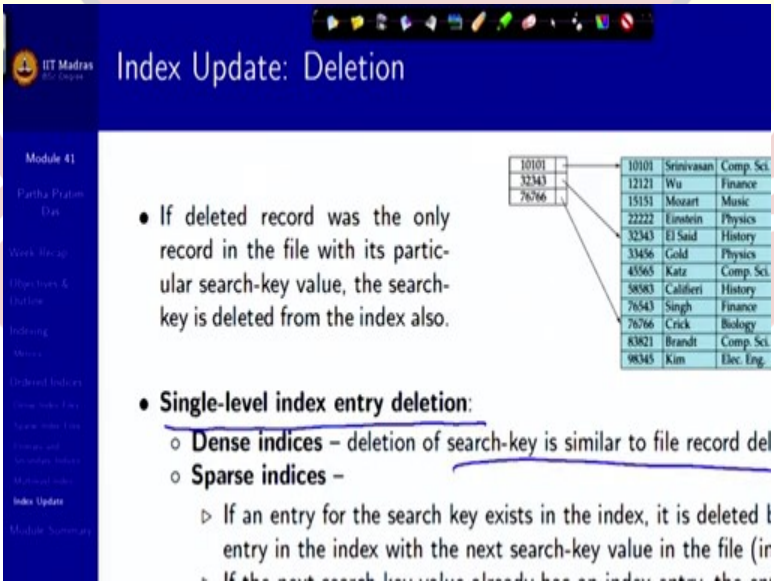
outer index may not fit in into the main memory, not even in couple of blocks and then you might need to index it again, so this could go to multiple levels as the database grows.

(Refer Slide Time: 25:13)



So, this is a general structure where you have, this is a outer index which indexes on the blocks of the inner index which in in turn indexes on the blocks of the file and this could go to multiple levels.

(Refer Slide Time: 25:31)

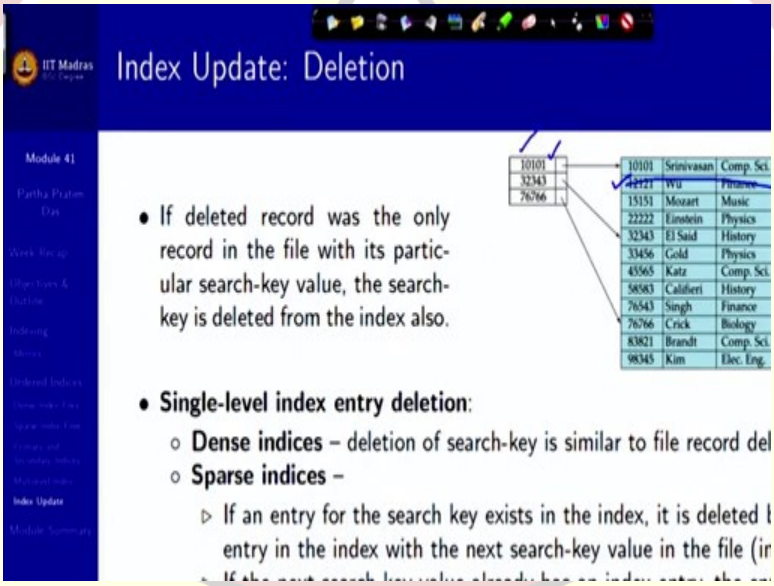


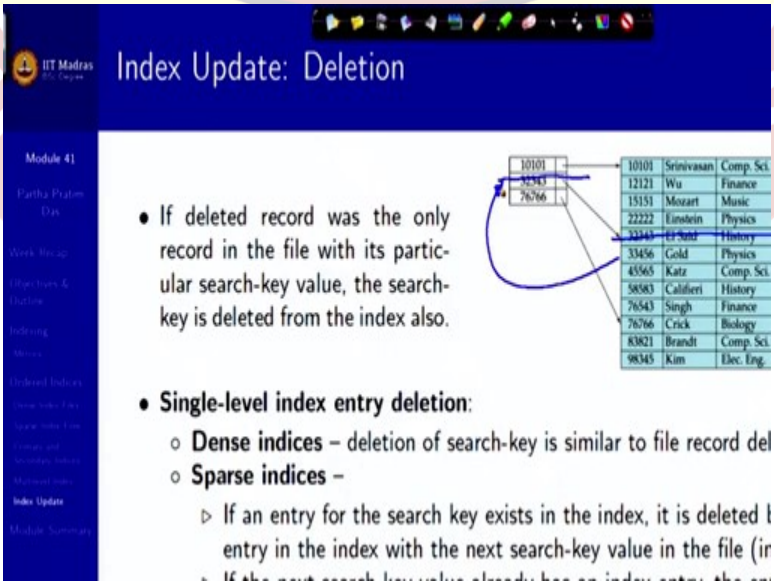
So, these are, this is another way I mean another mechanism by which you can handle the scalability of the database, the idea is nothing specifically novel but what you are handling is you

can let the database grow and grow and grow without substantial degradation of the performance.

Now, obviously if you have to update the index you have to take certain strategy, so if you delete a record that was the only record in the file with that particular search key value, the search key will also have to be deleted from that index, if it had duplicates then you will, you may not. Now, if you have single level index then to delete in terms of a dense index it will be similar to the deletion of the file record, because it is exactly the same, if it is sparse, then there could be several, if it is sparse like this, then there could be several differences.

(Refer Slide Time: 26:33)

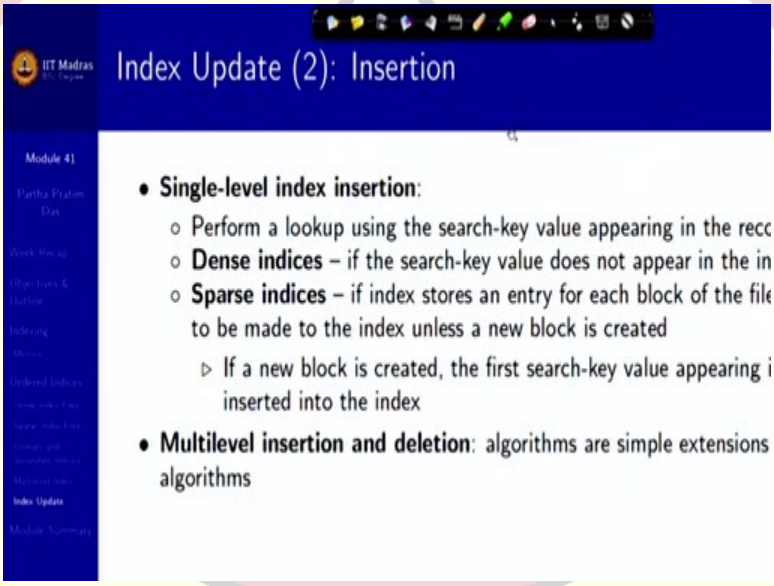




For example, a record being deleted for example if this is deleted it does not have an entry in the sparse index. But two bounding values have 10101 and 32343. So, you can remove this record without actually impacting this. But suppose if you delete, if you happen to delete this record, this is an entry in the sparse index table.

So, if you delete this then you will have to delete this here as well and what has to come in its place is the next one. So, these operations will be involved, so in the sparse index the deleted entry will have to be replaced and if it may also happen that that value is already there, if it is already there then you do not need to, would not need to do anything.

(Refer Slide Time: 27:35)

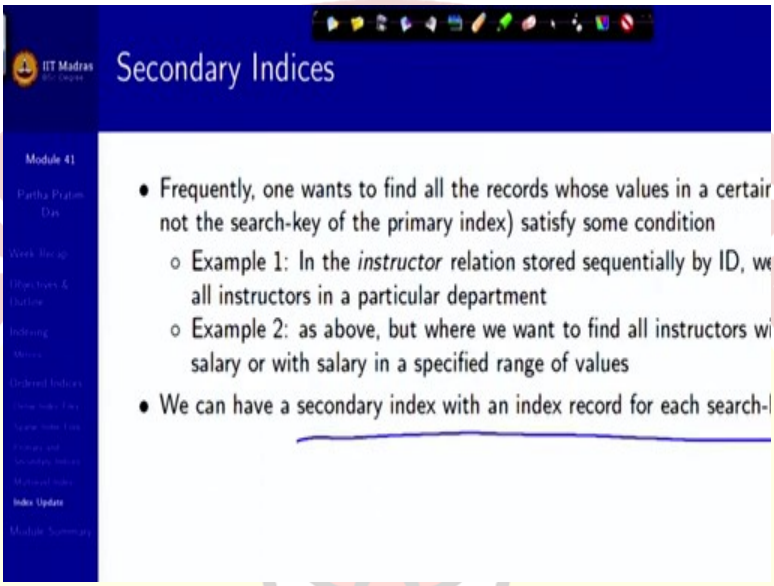


So, similarly for insertion you will have to take a similar strategy, perform a lookup using the search key, locating what where you can insert it. For dense index the search key does not, if the search key does not appear on the index you have to insert it, if it does because through the secondary index mechanism then you do not have to.

And in case of sparse index you will have to see what are the conditions, if it falls within already a range that is supported then you do not need to do anything but if you are doing a block level index and a new block is created, then the first search key of that new block will have to be entered, so in, it will branch out into multiple cases which are not difficult to reason out, but you will have to take care of all of these and that is how your index management could be efficiently done.

And that is the cost you are paying for the efficiency of the access. Naturally these all will extend in terms of multi-level insertion and deletions, but they are just simple extensions of the basic logic. So, you can always work them out.

(Refer Slide Time: 28:51)

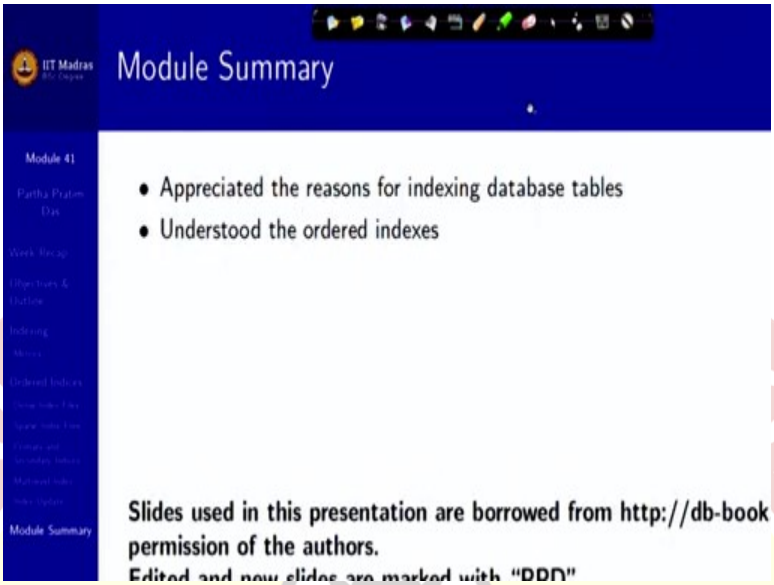


In case of a secondary index we often want to find all records whose values in a certain field satisfy some condition which is not the search key of the primary index. For example, I want the instructor relation is stored sequentially by ID, but we may want to find all instructors in a particular department or we want to find all instructors in a particular department but where you want all instructors with specified salary or with a specified salary range.

So, these are kind of situations where a lot of linear search may get involved. So, if this is a very typical query you might want to create a secondary index based on these values. So, that you can quickly come to that bundle of values on which you have to do the next level of filter. So, all of it, the bottom line is as we have understood a linear search is order n, a binary search is order log n.

So, if I can search within sorted values then I benefit very very significantly, otherwise I will have to do a linear search. Now, given all the tradeoffs of space and additional update, insert, delete, overhead and so on, we cannot have this kind of a sorted search key available everywhere, we will have to analyze the queries and decide what and how we want to create the indexes.

(Refer Slide Time: 30:45)



And often what happens is when you do the design and decide on these are my indexes, then possibly over a period of time you will let the database operate with the different queries because you may know, you will certainly have to know the queries to have coded it, but you may not know, you will in general not know what is the statistical distribution of the query being asked, actually being asked, what is the probability of a certain query or what is the probability of certain type of range being asked for and so on so forth.

So, that is where you remember we talked about in the data dictionary mechanism, in the system catalog mechanism that there is a lot of feature available to collect statistical information. So, those statistical information is getting collected, so from time to time the database designer has to come back, look into those statistical information and see what are the queries that are getting frequently asked but are actually slow in the process, analyze that, readjust the index. So, that is the whole game all about. Thank you very much for your attention and we will continue in the next module.